

Acessibus - Requisitos das telas

☐ Tela de cadastro

▼ Objetivo

- Salvar as informações de um usuário no banco de dados utilizando validação e criptografia

▼ Interface

- Logo
- Input (*texto*) Nome
- Input (*texto*) Email
- Input (*texto*) Senha
- Input (*png ou jpeg*) Foto
- Botão para cadastrar
- (Opcional) Botão para cadastro com facebook
- (Opcional) Botão para cadastro com Google
- Menu
 - Botão para página Home
 - Botão para página recentes
 - Botão para página favoritos

▼ Regras de negócio

☐ Validação

- ☐ Nome, email e senha não podem ser nulos ou estar em branco
- ☐ Email deve ser único e ter 1 "@" e 1 "."
- ☐ Senha com tamanho min de 8 char e máx de 20
- ☐ Foto somente arquivos png ou jpeg

- ☐ Qualquer campo inválido impede o cadastro e retorna a mensagem de erro específica no front-end
- ☐ Criptografar senha utilizando hash
- ☐ Clicar em cadastrar gera um token e redireciona para tela inicial utilizando o token
- ☒ ~~Dados do usuário são salvos no banco~~

▼ Fluxos

- Rota: **POST /api/cadastro**
- Fluxo sucesso:
 - Usuário insere os dados
 - Front-end envia os dados para API
 - API valida e retorna token
 - Front-end salva token em asyncStorage
 - AuthContext logado = true
 - Usuário é redirecionado para página home
- Fluxo erro:
 - Usuário insere algum dado inválido
 - API retorna 400 bad request
 - Texto vermelho acima do input informando o erro
 - Erro interno
 - API retorna 500 internal server error
 - Alert informando erro

☐ Tela de login

▼ Objetivo

- Autenticar usuário e gerar token com base nos dados fornecidos após validação

▼ Interface

- Logo
- Input (*texto*) Email
- Input (*texto*) Senha
- Botão para logar
- (Opcional) Botão para login com facebook
- (Opcional) Botão para login com Google
- Menu
 - Botão para página Home
 - Botão para página recentes
 - Botão para página favoritos

▼ Regras de negócio

- ☐ Validação
 - ☐ Email existe no banco
 - ☐ Email recebeu texto com 1 "@" e 1 "."
 - ☐ O hash da senha inserida é o mesmo no banco para aquele email
- ☐ Gerar token se passar nas validações

▼ Fluxos

- Rota: **POST /api/auth/login**
- Fluxo sucesso:
 - Usuário insere os dados
 - Front-end envia os dados para API
 - API valida e retorna token
 - Front-end salva token em asyncStorage
 - AuthContext logado = true
 - Usuário é redirecionado para página home
- Fluxo erro:
 - Usuário insere algum dado inválido

- API retorna 400 bad request (dados faltando) OU
- API retorna 401 Unauthorized para email ou senha incompatíveis com os do banco (SEGURANÇA: NÃO INFORMAR SE É O EMAIL OU A SENHA QUE ESTÁ ERRADA)
- Texto vermelho acima do input informando o erro
- Erro interno
 - API retorna 500 internal server error
 - Alert informando erro

☐ Tela inicial

▼ Objetivo

- Funcionalidade principal. Focada em acessibilidade. Usuário utiliza comando de voz para informar destino ou linha desejada. O sistema transcreve o áudio e busca correspondências diretas no banco de dados.

▼ Interface

- Logo
- Renderização condicional:
 - SE usuário logado: botão para acessar o perfil
 - SENÃO: botão para fazer login
- Botão principal (Gigante e centralizado para acessibilidade tátil)
- Soundwave/Feedback sonoro indicando que está gravando
- Menu
 - Botão para página Home
 - Botão para página recentes
 - Botão para página favoritos

▼ Regras de negócio

- ☐ Integração com API de Transcrição (Whisper ou Google Speech) para converter o áudio em texto.
- ☐ **Busca por Palavra-Chave:** O texto transcrito é utilizado para fazer uma busca `LIKE` (contém) no banco de dados na tabela de linhas (nome da linha ou itinerário).
- ☐ O usuário deve solicitar apenas o destino, sem adições. O retorno será a linha que o leva até aquele destino.
- ☐ Se o texto transcrito for muito curto ou vazio, retornar erro pedindo para repetir.
- ☐ Se a busca retornar mais de uma linha, o sistema deve listar as opções.
- ☐ Verificar se existe relacionamento entre usuario e linha encontrada: se não, criar; se existir, atualizar coluna "ultimo_acesso".

▼ Fluxos

- Rota: POST `/api/search/audio` (Mudança: agora é um POST pois enviamos um arquivo de áudio)
- Fluxo sucesso:
 - Usuário aperta o botão principal.
 - Áudio é gravado e enviado para o Backend.
 - Backend envia áudio para API de Transcrição (Ex: OpenAI Whisper API).
 - API retorna texto (Ex: "Uniso").
 - Backend realiza query no banco:

```
SELECT * FROM linhas WHERE nome_linha LIKE '%Uniso%' OR
itinerario LIKE '%Uniso%'
```
 - Backend retorna o objeto da linha encontrada.
 - Front-end recebe o objeto e utiliza a funcionalidade nativa de leitura de tela (VoiceOver/TalkBack) ou uma API de TTS para falar: "A linha 52 - Cidade Universitária leva até a Uniso."
- Fluxo erro:
 - API de transcrição falhou ou não entendeu o áudio

- Retorno 400.
- Feedback sonoro: "Não consegui entender, por favor repita o nome da linha ou destino."
- Nenhuma linha encontrada no banco com aquele nome
 - Retorno 404.
 - Feedback sonoro: "Nenhuma linha encontrada com o nome [Texto Transcrito]."

☐ Tela recentes

▼ Objetivo

- Usuário pode ter acesso às últimas linhas que ele utilizou.

▼ Interface

- Logo
- Renderização condicional:
 - SE usuário logado: Últimas linhas acessadas ordenadas pela coluna "ultimo_acesso" de forma decrescente. Cada linha pode ser favoritada com um botão
 - SENÃO: Texto informando que é preciso estar logado para ter acesso às últimas linhas acessadas. Botão que redireciona para tela de login. Link que redireciona para tela de cadastro.
 - SE não existir relacionamento do usuário com alguma linha: texto mostrando que primeiro é necessário pesquisar uma linha para que ela seja adicionada aos recentes
- Menu
 - Botão para página Home
 - Botão para página recentes
 - Botão para página favoritos

▼ Regras de negócio

- ☐ Verificar se usuário está logado

☐ Pressionar botão de favoritos muda coluna "favorito" em "usuarios_linhas"

▼ Fluxos

- Rota: GET **/api/recents**
- Fluxo sucesso:
 - Verificação se usuário está logado
 - Verificação se existem relacionamentos do usuario com linhas
 - Ordenanação das linhas que se relacionam com usuario pelo ultimo_acesso de forma decrescente
 - Renderização dos 5 primeiros resultados
- Fluxo erro:
 - Usuário não está logado
 - API retorna 401 unauthorized
 - Renderização condicional redirecionando para tela de cadastro/login
 - Não existem relacionamentos entre o usuario e linhas
 - API retorna 200
 - Renderização condicional informando sobre os relacionamentos
 - Erro interno
 - API retorna 500 internal server error
 - Alert informando erro

☐ Tela favoritos

▼ Objetivo

- Usuário pode ter acesso às linhas que ele favoritou.

▼ Interface

- Logo

- Renderização condicional:
 - SE usuário logado: Linhas favoritas ordenadas pela linha.
 - SENÃO: Texto informando que é preciso estar logado para ter acesso às linhas favoritas. Botão que redireciona para tela de login. Link que redireciona para tela de cadastro.
 - SE não existir relacionamento do usuário com alguma linha OU nenhuma das linhas relacionadas está favorita: texto mostrando que primeiro é necessário favoritar uma linha.
- POSTERIORMENTE ~~Botão para adicionar linha aos favoritos: abre um pop-up com botão semelhante ao principal mas menor. Depois do usuário solicitar uma linha o pop-up é excluído e surge outro com as linhas mais próximas do que o usuário solicitou.~~
- Botão para excluir uma linha dos favoritos
- Menu
 - Botão para página Home
 - Botão para página recentes
 - Botão para página favoritos

▼ Regras de negócio

- ☐ Verificar se usuário está logado
- ☐ Pressionar botão de excluir linha dos favoritos muda coluna "favorito" para false
- ☐ POSTERIORMENTE ~~Ao usuário apertar para adicionar uma linha aos favoritos ocorre um fluxo semelhante ao da página inicial~~

▼ Fluxos

- Rota: GET **/api/favorites**
- Fluxo sucesso:
 - Verificação se usuário está logado
 - Verificação se existem relacionamentos do usuario com linhas
 - Verificação se linhas relacionadas ao usuário possuem "favorito" como true

- Linhas renderizadas ordenadas por "nome_linha"
- Fluxo erro:
 - Usuário não está logado
 - API retorna 401 unauthorized
 - Renderização condicional redirecionando para tela de cadastro/login
 - Não existem relacionamentos entre o usuario e linhas
 - API retorna 200
 - Renderização condicional informando sobre os relacionamentos
 - Erro interno
 - API retorna 500 internal server error
 - Alert informando erro

☐ Tela perfil

▼ Objetivo

- Usuário pode ter acesso ao próprio perfil e linhas relacionadas

▼ Interface

- Logo
- Foto de perfil do usuário
- Nome
- Renderização condicional:
 - SE existir relacionamento do usuário com alguma linha: Mostrar linhas relacionadas
 - SENÃO: Texto informando que não há nenhuma linha relacionada e que é preciso pesquisar uma linha para que um relacionamento seja criado
- Botão para editar perfil
- Menu

- Botão para página Home
- Botão para página recentes
- Botão para página favoritos

▼ Regras de negócio

- ☐ Ao abrir a tela verificar se existem relacionamentos entre usuario e alguma linha

▼ Fluxos

- Rota: GET **/api/user**
- Fluxo sucesso:
 - Verificação se existem relacionamentos do usuario com linhas
 - Renderização de todos os relacionamentos
 - Usuário clicar em editar perfil redireciona para tela de editar perfil
- Fluxo erro:
 - Não existem relacionamentos entre o usuario e linhas
 - API retorna 404
 - Renderização condicional informando sobre os relacionamentos
 - Erro interno
 - API retorna 500 internal server error
 - Alert informando erro

- ☐ Tela atualizar perfil

▼ Objetivo

- Usuário pode atualizar próprio perfil

▼ Interface

- Logo
- Input (*png ou jpeg*) Foto
- Input (*texto*) Nome

- Input (*texto*) Email
- Input (*texto*) Senha
- Botão para atualizar
- Menu
 - Botão para página Home
 - Botão para página recentes
 - Botão para página favoritos

▼ Regras de negócio

- ☐ Solicitar senha para atualizar o usuário
- ☐ Todos os campos exceto a senha estarão preenchidos, usuário pode alterar o que quiser
- ☐ Campos são validados da mesma forma que no cadastro
- ☐ Se email já existir no banco de dados e for diferente do atual não permitir atualização

▼ Fluxos

- Rota: PUT **/api/user**
- Fluxo sucesso:
 - Verificação se existem relacionamentos do usuario com linhas
 - Renderização de todos os relacionamentos
 - Usuário clicar em editar perfil redireciona para tela de editar perfil
- Fluxo erro:
 - Não existem relacionamentos entre o usuario e linhas
 - API retorna 404
 - Renderização condicional informando sobre os relacionamentos
 - Erro interno
 - API retorna 500 internal server error
 - Alert informando erro